```
}
```

Κώδικας 6.20 *Έλεγχος ισότητας πινάκων*

Στον κώδικα 6.20 ορίζουμε αρχικά τους πίνακες όπως δίνονται στην εκφώνηση. Στη συνέχεια αρχικοποιούμε τη μεταβλητή equal στην τιμή της έκφρασης tbl1.length == tbl2.length. Αν οι πίνακες δεν έχουν ίδιο μήκος, η equal γίνεται false. Στη συνέχεια η ροή δεν μπαίνει στον βρόχο for και απλά εμφανίζει την τιμή false. Αν όμως τα μήκη των πινάκων είναι ίσα μεταξύ τους, τότε η ροή εισέρχεται στον εξωτερικό βρόχο for. Εκεί ελέγχεται κάθε στοιχείο του tbl1 αν υπάρχει και στον tbl2. Όσο ένα στοιχείο του tbl1 βρίσκεται και στον tbl2, η βοηθητική μεταβλητή found γίνεται true και αντίστοιχα ενημερώνεται και η equal. Αν ένα στοιχείο του tbl1 δεν βρεθεί στον tbl2, η found γίνεται false και διακόπτεται η εσωτερική for. Ακολούθως γίνεται false η equal η οποία θα διακόψει και την εξωτερική for.

6.5.4 Αντιστροφή πίνακα

Να γράψετε κώδικα ο οποίος με δεδομένο τον πίνακα int[] tbl = {1, 2, 3, 14, 5, 6, 7, 8, 9} παράγει έναν πίνακα που περιέχει τα ίδια στοιχεία με τον tbl σε αντεστραμμένη διάταξη, δηλαδή στη θέση 0 έχει το 9 και στη θέση 8 το 1.

Λύση

```
public static void reverse() { //Κώδικας 6.21
    int[] tbl = {1, 2, 3, 14, 5, 6, 7, 8, 9};
    int[] reversed = new int[tbl.length];
    for (int i = 0; i < tbl.length; i++) {
        reversed[tbl.length - 1 - i] = tbl[i];
    }
    System.out.print(Arrays.toString(reversed));
}
```

Κώδικας 6.21 *Παραγωγή πίνακα με αντεστραμμένη διάταξη στοιχείων*

Καταρχάς δημιουργούμε τον πίνακα reversed με ίδιο μήκος με τον tbl. Στη συνέχεια τοποθετούμε στη θέση tbl.length - 1 – i του reversed το στοιχείο που βρίσκεται στη θέση i του tbl.

6.5.5 Συνένωση πινάκων

Να γράψετε κώδικα που με δεδομένους τους πίνακες int[] t1={1,2,3,4,5}, t2={6,7,8,9} παράγει έναν πίνακα που συνενώνει τους t1 και t2.

Λύση

```
public static void merge() { //Κώδικας 6.22
    int[] t1 = {1, 2, 3, 4, 5}, t2 = {6, 7, 8, 9};
    int[] rVal = new int[t1.length + t2.length];
    System.arraycopy(t1, 0, rVal, 0, t1.length);
    System.arraycopy(t2, 0, rVal, t1.length, t2.length);
    System.out.println(Arrays.toString(rVal));
}
```

Κώδικας 6.22 *Συνένωση πινάκων*

Αφού δηλώσαμε τους πίνακες t1 και t2, δημιουργούμε τον πίνακα rVal με μήκος ικανό να φιλοξενήσει τα δεδομένα των t1 και t2. Στη συνέχεια, με την arraycopy μεταφέρουμε τα δεδομένα από τους t1 και t2. Τέλος, τυπώνουμε τον rVal ώστε να δείξουμε πως η συνένωση έγινε σωστά.

6.5.6 Εφαρμογή της arraycopy

Να γράψετε κώδικα που καταρχάς ορίζει τον πίνακα

```
char[][] copyFrom = {
     {'d', 'e'},
     {'c', 'a', 'f', 'f', 'e', 'i', 'n'},
     {'a', 't', 'e', 'd'}
};
```

Στη συνέχεια, ορίστε έναν πίνακα, copyTo, ικανών διαστάσεων ώστε να μπορεί να φιλοξενήσει αντίγραφο του copyFrom. Στη θέση 2 του copyTo τοποθετήστε τους χαρακτήρες 'l', 'i', 'k', 'e'. Αντιγράψτε από τον copyFrom στον copyTo τους πίνακες στις θέσεις 0 και 1. Εμφανίστε τον copyTwo.

Λύση

```
public static void copy() { ////Κώδικας 6.23
    char[][] copyFrom = {
        {'d', 'e'},
        {'c', 'a', 'f', 'f', 'e', 'i', 'n'},
        {'a', 't', 'e', 'd'}
    };
    char[][] copyTo = new char[3][];
    copyTo[2] = new char[]{'l', 'i', 'k', 'e'};
    System.arraycopy(copyFrom, 1, copyTo, 0, 2);
    for (char[] aElement : copyTo) {
        System.out.println(Arrays.toString(aElement));
    }
}
```

Κώδικας 6.23 *Λύση της άσκησης 6*

6.5.7 Μεταβολή μεγέθους

Να υλοποιήσετε συνάρτηση στην οποία να δηλώνετε έναν μονοδιάστατο πίνακα ακέραιων 6 θέσεων, έστω t. Στη συνέχεια καταχωρήστε τους ακεραίους 1 έως 6 στον t. Ορίστε επίσης μια ακέραια μεταβλητή, έστω newSize. Μεταβάλλεται το μήκος του t σε newSize. Φροντίστε ώστε ο κώδικάς σας να λειτουργεί σωστά είτε το newSize είναι μεγαλύτερο από το 6 είτε μικρότερο. Τυπώστε τον t ώστε να διαπιστώσετε πως έχει το αναμενόμενο μήκος.

Λύση

```
1  static void resize() { //Κώδικας 6.24
2      int[] t={1,2,3,4,5,6};
3      int newSize=4;
4      int[] tmp=new int[newSize];
5      System.arraycopy(t, 0, tmp, 0, t.length<newSize?t.length:newSize);
6      t=tmp;
7      System.out.println(Arrays.toString(t));
8  }
```

Κώδικας 6.24 *Μεταβολή μεγέθους πίνακα*

Ορίζουμε τον t και την newSize σύμφωνα με την εκφώνηση. Στη συνέχεια δημιουργούμε έναν άλλο πίνακα με μήκος newSize. Αμέσως μετά αντιγράφουμε δεδομένα από τον t στον tmp. Προσοχή σε αυτό το σημείο. Αν το newSize είναι μεγαλύτερο ή ίσο από το t.length, τότε μπορούν να αντιγραφούν όλα τα δεδομένα του t. Αν όμως είναι μικρότερο, τότε θα πρέπει να αντιγραφούν μόνο όσα χωρούν. Στον κώδικα 6.24 ο έλεγχος

αυτός γίνεται με τη βοήθεια του τριαδικού τελεστή στη γραμμή 5. Στη γραμμή 6 εκχωρούμε την αναφορά tmp στην t. Στη συνέχεια η t δείχνει σε πίνακα μεγέθους newSize.

6.6 Ασκήσεις προς λύση

1. Να γράψετε κώδικα στον οποίο να ορίζετε τους δισδιάστατους πίνακες ακέραιων 3x4, t1 και t2. Στη συνέχεια να δημιουργήσετε ένα πίνακα sum τέτοιον ώστε sum[i][j]=t1[i][j]+t2[i][j]. Εμφανίστε τον sum και ελέγξτε την ορθότητα του κώδικα.

2. Να γράψετε κώδικα που υπολογίζει ποια γραμμή δισδιάστατου πίνακα πραγματικών έχει το μεγαλύτερο άθροισμα στοιχείων.

3. Να γράψετε κώδικα που υπολογίζει ποια στήλη δισδιάστατου πίνακα πραγματικών έχει το μεγαλύτερο άθροισμα στοιχείων.

4. Να γράψετε κώδικα που αρχικοποιεί πίνακα ακέραιων 10x5 με τυχαίες τιμές από 0 έως 8. Τα δεδομένα σε κάθε γραμμή του πίνακα αναπαριστούν τις ώρες που εργάστηκε ένας εργαζόμενος κάθε μια από τις 5 εργάσιμες ημέρες μιας εβδομάδας. Υπολογίστε το σύνολο των ωρών που εργάστηκε κάθε εργαζόμενος κατά τη διάρκεια της εβδομάδας. Εμφανίστε τα σύνολα ταξινομημένα κατά φθίνουσα σειρά.

5. Να γράψετε κώδικα που αρχικοποιεί έναν δισδιάστατο πίνακα ακέραιων σε τυχαίες τιμές 0 και 1. Να υπολογίσετε πόσα 0 και πόσα 1 υπάρχουν σε κάθε γραμμή του πίνακα και πόσα σε όλον τον πίνακα.

6. Σε έναν δισδιάστατο πίνακα από String καταχωρήστε σε κάθε γραμμή του ένα κράτος της Ευρώπης με την πρωτεύουσά του. Στη συνέχεια υποβάλλετε 10 ερωτήσεις στον χρήστη και αξιολογήστε τις απαντήσεις του. Κάθε ερώτηση μπορεί να δίνει την ονομασία του κράτους και να ζητά την πρωτεύουσα ή να δίνει την πρωτεύουσα και να ζητά το κράτος.

7. Να αναπτυχθεί συνάρτηση που λαμβάνει ως παράμετρο έναν πίνακα ακέραιων και επιστρέφει 1 αν ο πίνακας είναι ταξινομημένος κατ' αύξουσα σειρά, -1 εν είναι ταξινομημένος κατά φθίνουσα σειρά και 0 αν δεν είναι ταξινομημένος.

8. Να δηλώσετε έναν πίνακα, a, από συμβολοσειρές. Να εκχωρήσετε μια συμβολοσειρά σε κάθε θέση του πίνακα. Στη συνέχεια, να δημιουργήσετε ένα αντίγραφο του a τέτοιο ώστε κάθε αλλαγή σε μια συμβολοσειρά του a να μην επηρεάζει την αντίστοιχη συμβολοσειρά στο αντίγραφο.

Βιβλιογραφία

[1] C. Horstmann, Big Java: Early Objects, 7th Edition | Wiley, 7th ed. Laurie Rosatone, 2013. Accessed: Sep. 14, 2021. [Online]. Available: https://www.wiley.com/en-us/Big+Java%3A+Early+Objects%2C+7th+Edition-p-9781119499091

[2] D. Liang, Εισαγωγή στον Προγραμματισμό Java, 10η. Θεσσαλονίκη: Τζιόλας.

Κεφάλαιο 7

7. Στατικές Συναρτήσεις και Μεταβλητές

Είναι συχνό φαινόμενο το ίδιο τμήμα κώδικα να είναι αναγκαίο σε διάφορα σημεία ενός προγράμματος. Ένας απλοϊκός τρόπος για να αντιμετωπιστεί η αναγκαιότητα αυτή είναι να επαναλάβουμε το ίδιο τμήμα όπου και όσες φορές αυτό χρειάζεται. Όμως, η προσέγγιση αυτή παρουσιάζει σοβαρά μειονεκτήματα. Καταρχάς, καθιστά τη συντήρηση του κώδικα πολύ δύσκολη. Αν χρειαστεί να κάνω μια βελτίωση ή οποιαδήποτε άλλη μεταβολή στο τμήμα του κώδικα που επαναλαμβάνεται, τότε θα πρέπει να βρω όλα τα σημεία στα οποία ενσωματώνεται και να τα μεταβάλω ένα προς ένα. Φανταστείτε τώρα πόσο δυσχεραίνεται η συντήρηση του κώδικα αν στο πρόγραμμά σας έχετε πολλά τμήματα κώδικα που το καθένα χρειάζεται σε πολλά σημεία, κάτι που είναι πολύ συνηθισμένο στα σύγχρονα προγράμματα.

Ένα άλλο θέμα μάλλον σημαντικότερο και από τη δυσκολία στη συντήρηση είναι η έλλειψη αφαιρετικότητας (abstraction). Ας υποθέσουμε πως έχουμε ένα τμήμα κώδικα που υπολογίζει τη δύναμη ενός αριθμού υψωμένου σε μια δύναμη. Έστω κατά την ανάπτυξη της εφαρμογής μας, χρειαζόμαστε αυτόν τον υπολογισμό. Πού είναι το τμήμα του κώδικα που τον κάνει; Θα πρέπει να ψάξουμε. Έστω ότι το βρήκαμε. Ναι, όμως εκείνο το τμήμα υπολογίζει την τιμή της μεταβλητής x υψωμένης στην τιμή της μεταβλητής k. Σε ένα άλλο σημείο χρειαζόμαστε τον ίδιο υπολογισμό αλλά για διαφορετικές μεταβλητές. Πρέπει λοιπόν να συντηρούμε σε πολλά σημεία όχι απλώς τον ίδιο κώδικα αλλά περίπου τον ίδιο. Οι μεταβολές όμως από σημείο σε σημείο ενδέχεται να εισάγουν λάθη.

Σίγουρα θα ήταν βολικό να κατασκευάζαμε ένα υποπρόγραμμα που θα είχε κατάλληλη ονομασία, π.χ. power ή calcPower έτσι ώστε όποτε χρειαζόμασταν να υπολογίσουμε τη δύναμη ενός αριθμού, να το καλούσαμε να έκανε τους υπολογισμούς και να μας ενημέρωνε με το αποτέλεσμα. Ένα τέτοιο υποπρόγραμμα βέβαια δεν θα έπρεπε να υπολογίζει κάθε φορά τη δύναμη του ίδιου αριθμού υψωμένου στον ίδιο εκθέτη. Επομένως θα έπρεπε να υπάρχει τρόπος κάθε φορά που το καλούμε να το ενημερώνουμε σχετικά με τη βάση και τον εκθέτη του απαιτούμενου υπολογισμού.

Για καλή μας τύχη, τέτοιου είδους υποπρογράμματα υποστηρίζονται στην Java και ονομάζονται συναρτήσεις. Οι παράμετροι των συναρτήσεων είναι ο μηχανισμός μέσω του οποίου δίνουμε στη συνάρτηση τα δεδομένα στα οποία θα επενεργήσει ενώ ο κύριος τρόπος που η συνάρτηση μας επιστρέφει την απάντησή της είναι η τιμή επιστροφής της συνάρτησης.

Με αυτόν τον τρόπο επιτυγχάνουμε σημαντική αφαιρετικότητα στα προγράμματά μας. Δεν είναι αναγκαίο να γνωρίζουμε τις λεπτομέρειες του κώδικα μιας συνάρτησης για να τη χρησιμοποιήσουμε. Αρκεί να γνωρίζουμε τι κάνει (και όχι πώς το κάνει), το όνομά της, τις παραμέτρους που δέχεται και τι αποτελέσματα επιστρέφει. Έτσι είναι εύκολο να χρησιμοποιηθούν συναρτήσεις και μάλιστα από ένα ευρύτερο κοινό και όχι μόνο από αυτούς που τις έχουν υλοποιήσει.

Πέραν της τιμής επιστροφής της, μια συνάρτηση είναι δυνατό να παράγει αποτελέσματα και με άλλους τρόπους. Για παράδειγμα, μία συνάρτηση εισάγει σε έναν πίνακα μια σειρά από τιμές και επιστρέφει τον αριθμό των τιμών που εισήχθησαν. Τα αποτελέσματα που παράγει η συνάρτηση πέραν της τιμής επιστροφής της είναι γνωστά ως παράπλευρα αποτελέσματα (side effects). Η Java, όπως θα δούμε αμέσως παρακάτω σε αυτήν την ενότητα, υποστηρίζει συναρτήσεις που παράγουν μόνο παράπλευρα αποτελέσματα χωρίς να επιστρέφουν κάποια τιμή. Άλλες γλώσσες διαφοροποιούν τα υποπρογράμματα που παράγουν μόνο παράπλευρα αποτελέσματα από αυτά που επιστρέφουν κάποια τιμή. Για παράδειγμα, η Pascal ονομάζει τα υποπρογράμματα που δεν επιστρέφουν κάποια τιμή διαδικασίες (procedures), ενώ διατηρεί την ονομασία συναρτήσεις (functions) για τα υποπρογράμματα που επιστρέφουν τιμή.

Επάνω στη δυνατότητα να αναπτύσσουμε συναρτήσεις βασίζεται η δυνατότητα να οργανώνουμε τους πηγαίους κώδικες με δομημένο τρόπο. Δημιουργείται έτσι το μοντέλο του δομημένου προγραμματισμού (structured programming). Πρόκειται για ένα σημαντικό μοντέλο προγραμματισμού που διευκολύνει την ανάπτυξη και συντήρηση των προγραμμάτων, ενώ αποτελεί συστατικό στοιχείο του Αντικειμενοστρεφούς μοντέλου.

Η Java υποστηρίζει δύο ειδών συναρτήσεις, τις στατικές (static) και τις μη στατικές (non static). Το κύριο θέμα αυτού του κεφαλαίου είναι οι στατικές συναρτήσεις. Οι μη στατικές συναρτήσεις και μεταβλητές παρουσιάζονται αργότερα σε αυτό το εγχειρίδιο κατά την εισαγωγή στο Αντικειμενοστρεφές μοντέλο, στην ενότητα 11. Εδώ μαζί με την παρουσίαση των στατικών συναρτήσεων, παρουσιάζουμε και τις στατικές μεταβλητές της κλάσης.

7.1 Στατικές Συναρτήσεις

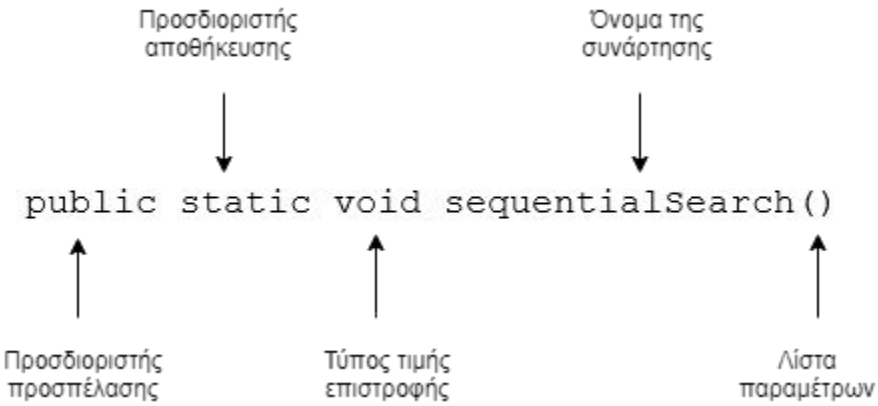
Έχουμε ήδη χρησιμοποιήσει στατικές συναρτήσεις στην πολύ απλή τους μορφή. Ας εξετάσουμε καταρχάς τη συνάρτηση sequentialSearch του κώδικα 6.12. Το τμήμα

```
public static void sequentialSearch()
```

ονομάζεται επικεφαλίδα της συνάρτησης (function header). Το μπλοκ που ακολουθεί την επικεφαλίδα ονομάζεται σώμα της συνάρτησης (function body). Η επικεφαλίδα της συνάρτησης ονομάζεται και διεπαφή της συνάρτησης (function interface), γιατί είναι το τμήμα τις λεπτομέρειες του οποίου είναι αναγκαίο να γνωρίζει ο χρήστης της συνάρτησης για να είναι σε θέση να την καλεί και να λαμβάνει την επιστροφή της συνάρτησης. Η επικεφαλίδα μαζί με το σώμα της συνάρτησης αποτελούν τον ορισμό της συνάρτησης (function definition). Ο ορισμός της συνάρτησης από μόνος του δεν παράγει κάποιο αποτέλεσμα σε ένα πρόγραμμα. Το αποτέλεσμα το παράγει η κλήση της συνάρτησης. Για παράδειγμα, η sequentialSearch χρειάζεται μια ακέραια μεταβλητή που ονομάζει schElement. Μνήμη για αυτήν τη μεταβλητή δεσμεύεται μόνον κατά την κλήση της συνάρτησης.

Στην αρχή της επικεφαλίδας έχουμε έναν προσδιοριστή προσπέλασης, στη συγκεκριμένη περίπτωση τον προσδιοριστή public. Για τους προσδιοριστές προσπέλασης έχουμε ήδη μιλήσει στην ενότητα 2.3. Στη συνέχεια ακολουθεί ο προσδιοριστής αποθήκευσης (class storage specifier), static που καθορίζει πως πρόκειται για στατική συνάρτηση. Μία στατική συνάρτηση μπορεί να κληθεί μέσω της κλάσης στην οποία ορίζεται όπως φαίνεται στον κώδικα 2.6.

Στη συνέχεια ακολουθεί η δεσμευμένη λέξη void που ορίζει πως η συνάρτηση αυτή δεν έχει τιμή επιστροφής. Μετά ακολουθεί το όνομα της συνάρτησης που είναι sequentialSearch και τέλος πριν το σώμα της συνάρτησης παρεμβάλλεται η λίστα παραμέτρων που στο παράδειγμά μας είναι κενή. Στο σχήμα 7.1 παρουσιάζονται τα συστατικά μέρη της διεπαφής της συνάρτησης.



Σχήμα 7.1 *Η διεπαφή της συνάρτησης*

Συνεχίζοντας την εξέταση της sequentialSearch παρατηρούμε πως αυτή ψάχνει να εντοπίσει μια συγκεκριμένη τιμή σε έναν συγκεκριμένο πίνακα. Μάλιστα τιμή αναζήτησης και πίνακας ορίζονται μέσα στην ίδια τη συνάρτηση. Αυτό σημαίνει πως η συνάρτηση αυτή δεν μπορεί να χρησιμοποιηθεί για κανέναν άλλο πίνακα αλλά ούτε και για να ψάξει άλλη τιμή έστω και στον ίδιο πίνακα. Ο αλγόριθμος όμως της σειριακής αναζήτησης είναι ίδιος για οποιονδήποτε μονοδιάστατο πίνακα και για οποιαδήποτε τιμή αναζήτησης. Επομένως, θα ήταν χρήσιμο να απομονώσουμε τον αλγόριθμο της σειριακής αναζήτησης κατά τρόπο ώστε να μπορούμε να τον εφαρμόζουμε σε οποιονδήποτε πίνακα και για οποιαδήποτε τιμή. Λύση σε αυτό το ζήτημα παρέχει η λίστα παραμέτρων της συνάρτησης. Η λίστα παραμέτρων δίνει τη δυνατότητα σε αυτόν που καλεί τη συνάρτηση να καθορίζει τόσο τον πίνακα όσο και την τιμή στα οποία επενεργεί η συνάρτηση. Όμως για να επιτευχθεί αυτό θα πρέπει κατά τον ορισμό της συνάρτησης να καθορίσουμε τους τύπους των παραμέτρων, όπως δείχνει παρακάτω η νέα διεπαφή της συνάρτησης.

```
public static void sequentialSearch(int[] array, int
schElement)
```

Η νέα διεπαφή επιβάλλει ώστε κατά την κλήση της συνάρτησης να δίνεται ένας πίνακας ακέραιων και μια ακέραια τιμή. Είναι προφανές πως κάποιες μεταβολές θα πρέπει να γίνουν στο σώμα της sequentialSearch. Ας δούμε όμως πρώτα πώς καλείται μια συνάρτηση με τη νέα διεπαφή της sequentialSearch.

```
int[] t1 = {3, 2, 5, 3, 6, 9};
int[] t2 = {6, 4, 5, 2, 1, 9, 9};
int sItem = 5;
sequentialSearch(t1, 9);
sequentialSearch(t2, sItem);
```

Κώδικας 7.1 *Κλήση της sequentialSearch με παραμέτρους, χωρίς τιμή επιστροφής*

Στον κώδικα 7.1 βλέπουμε δύο κλήσεις της sequentialSearch. Κατά την πρώτη κλήση η συνάρτηση αναζητά στον πίνακα t1 την τιμή 9 και στη δεύτερη κλήση αναζητά στον πίνακα t2 την τιμή της μεταβλητής sItem, δηλαδή την τιμή 5.

Ας σημειωθεί πως οι παράμετροι στον ορισμό της συνάρτησης ονομάζονται τυπικές παράμετροι (formal parameters), ενώ οι παράμετροι που δίνει ο χρήστης της συνάρτησης κατά την κλήση της ονομάζονται πραγματικές παράμετροι (actual parameters).

Το πρόβλημα στον σχεδιασμό της συνάρτησης όπως τον έχουμε τώρα είναι πως η συνάρτηση δεν επιστρέφει στον κώδικα που την καλεί το αποτέλεσμα της αναζήτησης. Αυτό όμως είναι αναγκαίο καθώς στο αποτέλεσμα της αναζήτησης, ο κώδικας που την καλεί θα βασίσει την περαιτέρω ροή του. Για παράδειγμα, ενδέχεται αν η τιμή βρεθεί, ο χρήστης της συνάρτησης να επιθυμεί να αυξήσει την τιμή στον

πίνακα κατά 1. Αν πάλι η τιμή δεν βρεθεί, μπορεί να μην χρειάζεται καμία περαιτέρω ενέργεια για τον συγκεκριμένο πίνακα.

Το πρόβλημα αυτό λύνεται με την τιμή επιστροφής της συνάρτησης. Στο συγκεκριμένο παράδειγμα μια καλή τιμή επιστροφής είναι τύπου int. Στην περίπτωση που θα βρεθεί η τιμή αναζήτησης, ο ακέραιος που θα επιστραφεί θα αναπαριστά τη θέση της τιμής αναζήτησης στον πίνακα. Αν πάλι η τιμή δεν βρεθεί, μπορεί να επιστραφεί η τιμή -1. Καθώς δεν υπάρχει θέση του πίνακα με αρνητική τιμή, η τιμή -1 μπορεί να αναπαραστήσει με ασφάλεια την αποτυχία της αναζήτησης. Προσέξτε εδώ πως αν διαλέξουμε να αναπαραστήσουμε την αποτυχία αναζήτησης με μη αρνητικό αριθμό, π.χ. με 0, τότε έχουμε κακό σχεδιασμό της συνάρτησης. Αυτό γιατί η συνάρτηση θα επιστρέφει 0, τόσο στην περίπτωση που το στοιχείο αναζήτησης δεν βρεθεί, όσο και στην περίπτωση που βρεθεί στη θέση 0 του πίνακα.

Επομένως, η διεπαφή της συνάρτησης τροποποιείται όπως δείχνει ο ακόλουθος κώδικας:

```
public static int sequentialSearch(int[] array, int
schElement)
```

Η τιμή επιστροφής έγινε int από void που ήταν στην προηγούμενη έκδοση. Τώρα η συνάρτηση μπορεί να κληθεί όπως δείχνει ο κώδικας 7.2:

```
static void callSeqSearch() { //Κώδικας 7.2
    int[] t1 = {3, 2, 5, 3, 6, 9};
    int sItem = 5;
    int idx = sequentialSearch(t1, sItem);
    if (idx > -1) {
        t1[idx]++;
    }
}
```

Κώδικας 7.2 *Κλήση της sequentialSearch με παραμέτρους και τιμή επιστροφής*

Στον κώδικα 7.2, η κλήση της συνάρτησης επιστρέφει τη θέση του πίνακα στην οποία βρέθηκε το sItem ή -1. Αν επιστρέψει -1, καμία ενέργεια επάνω στον πίνακα δεν γίνεται. Αν όμως επιστρέψει μη αρνητική τιμή, τότε προσαυξάνεται η τιμή στη θέση που επιστράφηκε κατά 1.

Ακόμη και αν κάνουμε τις αλλαγές που συζητήσαμε μέχρι τώρα, παραμένει ένα σοβαρό μειονέκτημα στη συνάρτηση. Αυτό συνίσταται στο γεγονός ότι η συνάρτηση εμφανίζει τα αποτελέσματα της αναζήτησης. Προσέξτε, το μειονέκτημα δεν είναι ότι επιστρέφει στον κώδικα που την καλεί τα αποτελέσματα της αναζήτησης αλλά ότι τα εμφανίζει χρησιμοποιώντας μάλιστα την println. Αυτό το μειονέκτημα πολλές φορές γίνεται δύσκολα κατανοητό στους αρχάριους προγραμματιστές. Πράγματι, αν κάνω έναν κώδικα που ψάχνει σε συγκεκριμένο πίνακα για να βρει μια συγκεκριμένη τιμή και το μόνο που θέλω είναι να μάθω αν η τιμή βρέθηκε και που, τότε δεν είναι πρόβλημα αν ο κώδικας εμφανίζει το αποτέλεσμα της αναζήτησης. Μια συνάρτηση όμως μπορεί να κληθεί σε ένα σημείο που θέλω να εμφανίσω το αποτέλεσμά της αλλά μπορεί να κληθεί και σε πολλά σημεία που δεν θέλω να το εμφανίζω αλλά απλώς το χρειάζομαι μέσα στον κώδικά μου. Για παράδειγμα, μπορεί την sequentialSearch να την καλέσω σε μια παραθυρική εφαρμογή. Σε αυτήν την περίπτωση η έξοδος με την println που είναι κατάλληλη μόνο για περιβάλλοντα γραμμής εντολών (command line) προφανώς θα μου καταστρέψει τη διεπαφή χρήστη (user interface).

Ο κώδικας 7.3 δίνει τον ορισμό της sequentialSearch που ενσωματώνει όλες τις παρατηρήσεις μέχρι εδώ.

```
public static int sequentialSearch(int[] array, int sItem) {
    for (int i = 0; i < array.length; i++) {
        if (array[i] == sItem) {
            return i;
        }
```

```
        }
        return -1;
    }
```

Κώδικας 7.3 *Συνάρτηση σειριακής αναζήτησης σε μονοδιάστατο πίνακα ακέραιων*

Η sequentialSearch σε αυτή τη μορφή μπορεί να κληθεί από οποιονδήποτε κώδικα για να αναζητήσει σε οποιονδήποτε μονοδιάστατο πίνακα ακέραιων οποιαδήποτε ακέραια τιμή. Προσέξτε πως η επιστροφή τιμής από τη συνάρτηση γίνεται με χρήση της δεσμευμένης λέξης return. Η λέξη return συντάσσεται με έκφραση που ο τύπος της είναι υποχρεωτικά ίδιος ή συμβατός με τον τύπο της συνάρτησης. Η return προκαλεί άμεση έξοδο από τη συνάρτηση. Σε συναρτήσεις τύπου void, η return συντάσσεται μόνη, ακολουθούμενη απλώς από τον χαρακτήρα ελληνικό ερωτηματικό (semicolon). Σε void συναρτήσεις, η return δεν είναι υποχρεωτική. Ωστόσο, σε κάποιες περιπτώσεις είναι χρήσιμη.

Οι παράμετροι των συναρτήσεων στην Java είναι παράμετροι θέσης (positional parameters). Αυτό σημαίνει πως κατά την κλήση συνάρτησης, οι πραγματικές παράμετροι αντιστοιχίζονται με τις τυπικές παραμέτρους ανάλογα με τη θέση που διατηρεί κάθε μία στην λίστα παραμέτρων.

Τέλος να επισημάνουμε πως οι συναρτήσεις στα πλαίσια του αντικειμενοστρεφούς προγραμματισμού ονομάζονται και μέθοδοι (methods).

7.2 Υπερφόρτωση Συναρτήσεων

Πάντως, είναι πιθανό να χρειάζεται να κάνουμε αναζήτηση τιμών και σε πίνακες που δεν είναι πίνακες ακέραιων. Ας υποθέσουμε πως χρειάζεται να κάνουμε αναζήτηση σε έναν πίνακα πραγματικών. Σε αυτήν την περίπτωση μπορούμε να υλοποιήσουμε μια άλλη συνάρτηση με παραμέτρους τύπου double[] και double. Το θέμα είναι πως παρότι αλλάζουν οι τύποι των δεδομένων, η λειτουργία παραμένει ίδια. Δεν θέλουμε για την ίδια λειτουργία να έχουμε συναρτήσεις με διαφορετικό όνομα. Παρόλα αυτά, για κάποιες γλώσσες προγραμματισμού, η μόνη λύση είναι να κάνουμε μια συνάρτηση που έχει διαφορετικό όνομα από τη συνάρτηση που ψάχνει σε πίνακες ακέραιων. Αυτό όμως δεν είναι βολικό γιατί δεν θέλουμε να αναγκαζόμαστε να εφευρίσκουμε και να θυμόμαστε διαφορετικά ονόματα για την ίδια λειτουργία. Ευτυχώς, η Java όπως και οι περισσότερες σύγχρονες γλώσσες προγραμματισμού, επιλύουν αυτό το θέμα με την υπερφόρτωση συναρτήσεων (function overloading).

Με την υπερφόρτωση συναρτήσεων μπορούμε να ορίσουμε συναρτήσεις με το ίδιο όνομα αρκεί να διαφέρει η ταυτότητά τους (function signature). Η ταυτότητα μιας συνάρτησης αποτελείται από το όνομα της συνάρτησης και τη λίστα παραμέτρων. Προσέξτε πως η τιμή επιστροφής δεν συμπεριλαμβάνεται στην ταυτότητα. Επομένως, μπορούμε να κάνουμε συναρτήσεις με το ίδιο όνομα, αρκεί να διαφέρει η λίστα παραμέτρων. Ποια από δύο συναρτήσεις που έχουν το ίδιο όνομα θα κληθεί είναι μια απόφαση που λαμβάνει ο μεταγλωττιστής ανάλογα με τις πραγματικές παραμέτρους. Στον κώδικα 7.4 δίνουμε παράδειγμα ορισμού και χρήσης υπερφορτωμένων συναρτήσεων.

```
static void f(int i) { //Κώδικας 7.4
      System.out.println("f(int)");
}

static void f(double d) { //Κώδικας 7.4
      System.out.println("f(double)");
}

public static void main(String[] args) { //Κώδικας 7.4
      f(1);
      f(1d);
}
```

Κώδικας 7.4 *Ορισμός και χρήση υπερφορτωμένων συναρτήσεων*